

Aufgabe 2: Planung der Tests

Das vorliegende Dokument beschreibt die Testplanung für das Projekt Prekaj-Zeiterfassung. Ausgehend von den identifizierten Produktrisiken und dem Qualitätsanspruch der Anwender werden Testziele abgeleitet und ein maßgeschneiderter Testansatz für Unit- und Systemtests sowie die benötigte Testinfrastruktur festgelegt.

1. Relevante Produktrisiken

Im Folgenden werden die wesentlichen Produktrisiken für den Einsatzzweck der Prekaj-Zeiterfassung im Familienbetrieb FliesenPrekaj GmbH dargestellt. Die Bewertung erfolgt aus der Perspektive „Was passiert im schlimmsten Fall, wenn die Software einen Fehler hat?“ und bildet die Grundlage für die nachfolgende Priorisierung der Testmaßnahmen.

Risiko	Mögliche Auswirkung	Schaden	Wahrscheinlichkeit
Falsche Erfassung von Arbeitszeiten	Zu viele Überstunden → unberechtigt höherer Lohn für den Inhaber; zu wenige Stunden → finanzieller Nachteil für den Arbeiter	Hoch	Mittel
Fehlerhafte Authentifizierung	Ein Arbeiter kann auf die Daten eines anderen Arbeiters zugreifen; DSGVO-Verstoß	Hoch	Niedrig
Keine Internetverbindung auf der Baustelle	Arbeiter kann sich nicht einstempeln/ausstempeln, obwohl er muss	Mittel	Hoch
Fehlerhaftes oder nicht konformes PDF-Reporting	Monatsbericht / Lohnzettel unbrauchbar – stellt den gesamten Zweck der App in Frage; Probleme bei Vorlage beim Finanzamt	Hoch	Mittel
Urlaubsanträge gehen verloren oder werden doppelt verbucht	Konflikte zwischen Arbeiter und Inhaber („Ich habe doch vor zwei Wochen einen Antrag gestellt.“); falsche Urlaubsstände	Mittel	Niedrig

Die Risiken rund um Zeiterfassung und PDF-Reporting werden als die wirtschaftlich kritischsten eingestuft, da sie direkt in die Lohnabrechnung und in die Belege für das Finanzamt einfließen. Authentifizierungsfehler sind aufgrund der DSGVO-Relevanz ebenfalls hoch priorisiert.

2. Qualitätsanspruch der Anwender

Die Erwartungshaltung der beiden Anwendergruppen unterscheidet sich deutlich. Aus diesen Erwartungen ergeben sich die zentralen Qualitätsmerkmale, an denen sich die Tests orientieren.

Arbeiter (Mobile-App)

- Möchte Arbeitsbeginn und -ende einfach und schnell eintragen können, ohne dass die App ihn an der eigentlichen Arbeit hindert.
- Möchte den eigenen Urlaubsanspruch jederzeit transparent einsehen können.
- Möchte die eigenen Überstunden nachvollziehbar einsehen können.
- Erwartet, dass die App auch unter realen Baustellenbedingungen (instabile Internetverbindung) zuverlässig funktioniert.

Inhaber / Admin (Web-App)

- Erwartet, dass die Aufzeichnungen der Arbeitszeiten vollständig und korrekt sind.
- Erwartet einen übersichtlichen Zugriff auf alle relevanten Informationen, sodass diese schnell auffindbar sind.
- Erwartet Nachvollziehbarkeit, welcher Arbeiter wie lange an welchem Bauvorhaben tätig war.
- Erwartet, dass die PDF-Berichte – insbesondere solche, die dem Finanzamt vorgelegt werden – vollständig und korrekt formatiert sind.

3. Abgeleitete Testziele

Aus den Risiken und Qualitätsansprüchen ergeben sich die folgenden Testziele. Jedes Testziel ist auf mindestens ein Produktrisiko bzw. einen Qualitätsanspruch zurückführbar und damit prüfbar.

1. Nachweisen, dass der Beginn und das Ende einer Arbeitsaufzeichnung mit Datum, Stunde, Minute und Sekunde korrekt erfasst und persistent gespeichert werden.
2. Nachweisen, dass nach einem Arbeitstag mit mehr als 6 Stunden Arbeitszeit die gesetzlich vorgeschriebene Pause von 30 Minuten korrekt von der Arbeitszeit abgezogen wird.
3. Nachweisen, dass jeder authentifizierte Benutzer ausschließlich auf die ihm zugeordneten Daten zugreifen kann.
4. Nachweisen, dass ein Arbeiter unter keinen Umständen die Daten eines anderen Arbeiters einsehen oder verändern kann.
5. Nachweisen, dass die generierten PDF-Berichte (Monatsbericht, Lohnzettel) den Datenbestand in der Datenbank vollständig und korrekt wiedergeben.

4. Testansatz Unittests

Unittests bilden die unterste Teststufe und prüfen einzelne Code-Einheiten isoliert. Schwerpunkt liegt auf der Geschäftslogik des Backends, da hier die wirtschaftlich kritischen Berechnungen stattfinden.

Testbasis	Für die Berechnung der Arbeitszeit gelten u. a. folgende Regeln, die als Testbasis dienen: • Soll-Stunden: Bestandteil des Arbeiter-Stammdatenmodells (z. B. Vollzeit = 40 h/Woche = 8 h/Tag → 8:30 h Anwesenheit aufgrund 30 Min. gesetzlicher Pause). • Tatsächliche Stunden = Ende – Anfang einer Arbeitsaufzeichnung. • Ist-Stunden = Soll-Stunden – Tatsächliche Stunden (Basis für die Überstunden-/Minusstundenberechnung). Für die Authentifizierung dienen technische und sicherheitsbezogene Spezifikationen (eigener Auth-Service im Backend) als Testbasis.
Testobjekte	Einzelne Go-Funktionen und Packages des Backends mit Fokus auf die Geschäftslogik: • Package „timetracking“ – Erfassung von Start-/Endzeiten, Berechnung der tatsächlichen Arbeitszeit. • Package „overtime“ – Berechnung von Über- und Minusstunden inkl. Berücksichtigung der gesetzlichen Pause. • Package „auth“ – Login, Token-Validierung, rollenbasierte Zugriffsprüfung. • Hilfsfunktionen für die PDF-Datenaufbereitung.
Geplante Testabdeckung	Mindestens 80 % Line-Coverage für die geschäftskritischen Packages „timetracking“ und „overtime“, da Fehler hier unmittelbar in die Lohnabrechnung einfließen. Für das Package „auth“ wird eine vergleichbare Abdeckung angestrebt, ergänzt um gezielte Negativtests (z. B. ungültige Tokens, fremde Benutzer-IDs). Reiner Glue-Code und einfache Datenbank-Adapter werden best-effort getestet, ohne fixe Quote.
Verwendetes Testwerkzeug	• Standard-Package „testing“ aus der Go-Standardbibliothek. • „testify“ für Assertions und Mocks. • „go test -cover“ zur Messung der Code-Abdeckung.

5. Testansatz Systemtests

Systemtests prüfen das Gesamtsystem aus Anwendersicht. Da Web- und Mobile-Frontend ausschließlich über die Backend-API kommunizieren, bildet die API den zentralen, gut testbaren Integrationspunkt. Auf den Frontends werden ergänzend End-to-End-Tests durchgeführt.

Testbasis	• Use Cases aus Abschnitt 3 der Projektbeschreibung (Zeiterfassung starten/beenden, Lohnzettel herunterladen, Urlaubsantrag stellen, Monatsbericht erstellen, Urlaubsanträge verwalten, Überstunden überwachen, ...). • Anforderungen aus den Projektzielen (insb. monatliche Übersicht, automatisches Überstunden-Handling, DSGVO-Konformität). • Spezifikation der Backend-REST-API.
Testobjekte	Die laufende Anwendung als Black-Box auf zwei Ebenen: • Backend-API als zentraler Integrationspunkt – End-to-End-Tests auf HTTP-Ebene gegen eine Test-Instanz mit eigener PostgreSQL-Datenbank. • Web-Frontend (Admin) – End-to-End-Tests aus Sicht des Inhabers. • Mobile-Frontend (Arbeiter) – End-to-End-Tests aus Sicht des Arbeiters auf einem realen Gerät bzw. Emulator.
Geplante Testabdeckung	Maßgeblich ist die Anwendungsfall-Abdeckung, nicht die Code-Coverage: • Alle Use Cases aus Abschnitt 3 der Projektbeschreibung werden mit mindestens einem positiven und einem negativen Pfad abgedeckt. • Kritische Use Cases (Zeiterfassung starten/beenden, Monatsbericht erstellen, Urlaubsantrag stellen/verwalten) werden zusätzlich mit Edge Cases geprüft (z. B. Monatswechsel, Pausenregelung, Antrag über Jahreswechsel). • Sicherheitsrelevante Pfade (Zugriff auf fremde Daten, abgelaufene Sessions) werden gezielt getestet.
Verwendetes Testwerkzeug	• Backend-API: Postman / Newman für regressionsfähige API-Test-Collections; ergänzend Go-Integrationstests gegen eine Test-Datenbank. • Web-Frontend: Playwright (TypeScript) für End-to-End-Tests, da im React-/TypeScript-Stack gut integrierbar. • Mobile-Frontend: Maestro für End-to-End-Tests der React-Native-App, da deklarativ und mit geringem Setup-Aufwand nutzbar.

Strategie Automatisierung	• API-Tests werden vollständig automatisiert – hoher ROI durch Stabilität und schnelle Ausführbarkeit. • Kritische End-to-End-Pfade (Stempeln, Monatsbericht, Urlaubsfreigabe) werden automatisiert, ergänzt durch manuelle Tests für visuelle und explorative Aspekte. • Eine CI/CD-Pipeline ist aktuell noch nicht im Einsatz (vgl. Projektbeschreibung Abschnitt 7). Tests werden daher zunächst lokal angestoßen; mit Aufbau der CI/CD-Pipeline werden die automatisierten Tests dort als verpflichtender Schritt integriert. • Vor dem produktiven Einsatz (Meilenstein M7) sind zusätzlich Abnahmetests mit dem Arbeiter und dem Inhaber vorgesehen, um die Praxistauglichkeit aus Sicht der realen Anwender zu bestätigen.
----------------------------------	--

6. Benötigte Testinfrastruktur

Die Testinfrastruktur stellt die organisatorischen und technischen Voraussetzungen für die Durchführung der oben beschriebenen Tests sicher.

Testumgebung	Vom Produktivbetrieb getrennte, produktionsnahe Umgebung: • Lokale Docker-Compose-Umgebung auf dem Entwickler-Rechner mit eigener PostgreSQL-Instanz für die schnelle Test-Iteration. • Geplante Test-Stage auf einem separaten lokalen Firmenserver, die später aus der CI/CD-Pipeline heraus automatisch bereitgestellt und mit Testdaten befüllt wird. • Strikte Trennung zur Produktivdatenbank, um den Echtbetrieb und personenbezogene Daten zu schützen (DSGVO).
Testdaten	Ausschließlich synthetische, DSGVO-unbedenkliche Testdaten: • Test-Arbeiter mit unterschiedlichen Arbeitszeitmodellen (z. B. Vollzeit, Teilzeit). • Zeitbuchungen über mehrere Monate hinweg, inkl. Spezialfälle wie Monatswechsel, Schaltjahr, Wochenenden, lange Schichten. • Urlaubsanträge in unterschiedlichen Zuständen (offen, freigegeben, abgelehnt) und mit Sonderfällen (Antrag über Jahreswechsel, sich überschneidende Anträge). • Bereitstellung über Seed-Skripte und Fixtures, sodass jede Testumgebung reproduzierbar in einen definierten Ausgangszustand versetzt werden kann.
Werkzeuge	• Go „testing“, „testify“, „go test -cover“ – Unittests und Coverage. • Postman / Newman, Go-Integrationstests – API- und Backend-Integrationstests.

- Playwright – End-to-End-Tests Web-Frontend.
- Maestro – End-to-End-Tests Mobile-Frontend (React Native).
- Docker / Docker Compose – Bereitstellung der Testumgebung.
- Git – Versionierung von Testcode und Testdaten-Seeds.
- Geplant: golangci-lint für statische Codeanalyse sowie eine CI-Plattform (z. B. GitLab CI oder GitHub Actions) für die spätere Pipeline-Integration.